

Facebook Comments Workflow Enhancement

A comprehensive analysis of the /facebook/comments route in Whatomate, identifying 15 logic gaps and proposing 8 workflow enhancements across tenancy, reliability, async processing, and frontend consistency.

Project: Whatomate

Route: /facebook/comments

Date: June 2026

Author: Z.ai Code Review Agent

Table of Contents

1. Executive Summary	2
2. Current Architecture Overview	2
3. Identified Logic Gaps	3
4. Proposed Workflow Enhancements	5
4.1 Enhancement 1: Async Auto-Reply via Queue Worker	5
4.2 Enhancement 2: Tenant-Scoped Page-to-Account Lookup	5
4.3 Enhancement 3: Unified Auto-Reply Settings Resolution	5
4.4 Enhancement 4: Error Logging for Silent Failures	6
4.5 Enhancement 5: Tenant Scope All Direct DB Access	6
4.6 Enhancement 6: Cursor-Based Pagination and Bounded Replies	6
4.7 Enhancement 7: Status Transition Validation	6
4.8 Enhancement 8: Frontend Conflict Resolution	7
5. Gap-to-Enhancement Mapping	7
6. Proposed Enhanced Sequence	8
7. Implementation Priority Order	9
8. Risk Assessment	9
9. Files Affected by Enhancement	10
10. Conclusion	11

1. Executive Summary

This document presents a comprehensive technical review and enhancement plan for the /facebook/comments route in the Whatomate platform. The review was conducted using multi-tool analysis: codebase-memory-mcp for pattern discovery, Socraticode for semantic code understanding and impact analysis, and Serena for precise source-level inspection.

15	8	6	1,680
Logic Gaps Identified	Proposed Enhancements	Sequence Changes	Lines of Code Reviewed

Figure 1: Review metrics at a glance.

The analysis covers the full webhook lifecycle: Facebook Graph API ingestion, HMAC-SHA256 signature verification, comment upsert with GORM, synchronous auto-reply (comment reply + private DM), WhatsApp notification bridging, WebSocket real-time broadcasting, and frontend merge logic for live updates. Key risk areas include cross-tenant data leakage in account lookup, silent error suppression in reply operations, and behavioral inconsistencies between the webhook and sync ingestion paths.

2. Current Architecture Overview

The Facebook comments feature is implemented primarily in internal/handlers/fb_comments.go (approximately 1,680 lines) with supporting models in internal/models/fb_comment.go. The route group is registered under /facebook/comments in cmd/whatomate/main.go with CSRF exemption and rate limiting applied at the webhook endpoint.

2.1 Request Flow

The webhook endpoint POST /facebook/comments/webhook receives real-time comment events from the Facebook Graph API. The processing pipeline follows this sequence:

Step	Operation	Function	Scope
1	HMAC-SHA256 signature verification	verifyFacebookWebhookSignature	Request

2	Parse webhook entry and extract page ID	ReceiveFacebookCommentsWebhook	Request
3	Lookup Facebook account by page ID	findFacebookAccountByPageID	Unscoped
4	Upsert comment into database	upsertFacebookWebhookComment	Org-scoped
5	Evaluate auto-reply eligibility	shouldAutoReplyFacebookComment	Org-scoped
6	Send comment reply + private DM	sendAndStoreFacebookCommentReply	Org-scoped
7	Bridge notification to WhatsApp	maybeSendWhatsAppCommentNotification	Unscoped
8	Broadcast via WebSocket	WSHub.Broadcast	Org-scoped

Table 1: Webhook processing pipeline sequence.

2.2 Data Models

Four GORM models support the feature, all in `internal/models/fb_comment.go`:

Model	Purpose	Key Fields
FacebookComment	Primary comment entity	22 fields: ID, Message, CommenterName, IsAdminReply, AutoRepliedAt, Metadata (JSONB)
FacebookCommentReply	Auto/manual reply records	13 fields: GraphCommentReplyID, GraphPrivateReplyID, Status
FacebookCommentSettings	Org-level settings	14 fields: AutoReplyEnabled, PrivateReplyEnabled, WhatsAppNotifyEnabled
FacebookPageCommentSettings	Per-page overrides	14 fields: WhatsAppInstanceID, page-specific toggle flags

Table 2: Facebook comment data models.

3. Identified Logic Gaps

The following table summarizes all 15 logic gaps discovered during the review, categorized by severity and domain. Each gap maps to one or more proposed enhancements in Section 4.

#	Gap	Severity	Domain	Location
---	-----	----------	--------	----------

1	Synchronous auto-reply blocks webhook response	High	Reliability	ReceiveFacebookCommentsWebhook
2	findFacebookAccountByPageID scans all orgs unscoped	Critical	Tenancy	findFacebookAccountByPageID
3	No deduplication guard for rapid re-ingest	Medium	Reliability	upsertFacebookWebhookComment
4	Silent error suppression in reply DB updates	High	Observability	sendAndStoreFacebookCommentReply
5	No retry/backoff on Facebook API failures	Medium	Reliability	sendAndStoreFacebookCommentReply
6	sendFacebookPrivateReply fails silently	High	Observability	sendFacebookPrivateReply
7	Sync path ignores page-level settings for auto-reply	Medium	Consistency	syncFacebookPageComments

Table 3: Logic gaps 1-7 (critical and high severity focus).

#	Gap	Severity	Domain	Location
8	Offset-based pagination degrades at scale	Low	Performance	ListFacebookComments
9	Unbounded reply preload in list queries	Medium	Performance	ListFacebookComments
10	No status transition validation	Medium	Data Integrity	UpdateFacebookCommentStatus
11	WhatsApp bridge uses a.DB unscoped	High	Tenancy	maybeSendWhatsAppCommentNotification
12	ReplyFacebookComment uses a.DB not requestDB	High	Tenancy	ReplyFacebookComment
13	pickRandomReplyText has no test coverage	Low	Testing	pickRandomReplyText
14	Webhook verifies but responds 200 even on AppSecret empty	Medium	Security	verifyFacebookWebhookSignature
15	Frontend merge logic lacks conflict resolution	Low	Frontend	facebookCommentsMerge.ts

Table 4: Logic gaps 8-15 (performance, consistency, and frontend).

4. Proposed Workflow Enhancements

Eight targeted enhancements are proposed to address the identified gaps. Each enhancement is designed to be backward-compatible and can be implemented incrementally. They are prioritized by blast radius: tenancy fixes first, then reliability, then performance.

4.1 Enhancement 1: Async Auto-Reply via Queue Worker

Move auto-reply logic from the synchronous webhook handler to an async Redis Streams worker. This directly addresses Gap 1 (blocked webhook response) and Gap 5 (no retry mechanism). The webhook handler will enqueue a job and return immediately, while the worker processes replies with exponential backoff.

Sequence change:

Current: webhook -> upsert -> auto-reply (sync) -> respond 200

Proposed: webhook -> upsert -> enqueue job -> respond 200; worker -> dequeue -> auto-reply (with retry)

Addresses: Gap 1, Gap 5

Priority: High (webhook latency and reliability)

4.2 Enhancement 2: Tenant-Scoped Page-to-Account Lookup

Add an org-scoped uniqueness constraint on page-to-account mapping in `findFacebookAccountByPageID`. Currently the function scans ALL active OAuth accounts across all organizations, creating a cross-tenant data leakage risk. The fix adds a Redis cache with org-prefixed keys and restricts DB queries to the identified tenant scope.

Additionally, a database unique index on (`organization_id`, `facebook_page_id`) should be added to prevent duplicate page registrations across tenants.

Addresses: Gap 2

Priority: Critical (multi-tenant security boundary)

4.3 Enhancement 3: Unified Auto-Reply Settings Resolution

Unify the auto-reply decision logic between `syncFacebookPageComments` (sync path) and `ReceiveFacebookCommentsWebhook` (webhook path). Currently the sync path checks only org-level settings and ignores per-page overrides, creating behavioral inconsistency. Both paths should use a shared `resolveAutoReplySettings()` helper that applies page-level overrides on top of org-level defaults.

Addresses: Gap 7

Priority: Medium (behavioral consistency)

4.4 Enhancement 4: Error Logging for Silent Failures

Replace all silent error suppression patterns (`_ = db.Updates(...).Error`) with explicit error logging via `slog.Error`. This affects `sendAndStoreFacebookCommentReply` (Gap 4) and `sendFacebookPrivateReply` (Gap 6). Errors should be logged with structured context: `org_id`, `comment_id`, `page_id`, and the specific operation that failed.

Addresses: Gap 4, Gap 6

Priority: High (operational visibility)

4.5 Enhancement 5: Tenant Scope All Direct DB Access

Replace all direct `a.DB` usage with `a.requestDB(rc)` or `tenant.ScopedDB()` in the following handlers: `ReplyFacebookComment` (Gap 12), `maybeSendWhatsAppCommentNotification` (Gap 11). These bypass the multi-tenant boundary and could expose data across organizations.

Addresses: Gap 11, Gap 12

Priority: Critical (multi-tenant data isolation)

4.6 Enhancement 6: Cursor-Based Pagination and Bounded Replies

Replace offset-based pagination in `ListFacebookComments` with cursor-based pagination (using comment ID or `created_at` as cursor) for stable results at scale (Gap 8). Add a `LIMIT` on the reply preload clause to prevent unbounded N+1 queries (Gap 9). A maximum of 50 replies per comment should be preloaded, with a separate endpoint for full reply history.

Addresses: Gap 8, Gap 9

Priority: Low (performance at scale)

4.7 Enhancement 7: Status Transition Validation

Add a finite state machine for comment status transitions in `UpdateFacebookCommentStatus`. Valid transitions should be defined (e.g., `new -> open -> resolved -> closed`) and the handler should reject invalid transitions with a clear error message. This prevents data integrity issues where comments skip states or revert from terminal states.

Addresses: Gap 10

Priority: Medium (data integrity)

4.8 Enhancement 8: Frontend Conflict Resolution

Enhance the WebSocket merge logic in `facebookCommentsMerge.ts` with last-write-wins conflict resolution based on `updated_at` timestamps. When a WebSocket update arrives for a comment currently being edited by the user, show a visual indicator that the comment has been modified externally. This addresses Gap 15 (no conflict resolution in merge).

Addresses: Gap 15

Priority: Low (user experience)

5. Gap-to-Enhancement Mapping

The following matrix maps each identified gap to its corresponding enhancement(s), providing full traceability from problem to solution.

Gap	Description	Enhancement
1	Sync auto-reply blocks webhook	4.1 Async Auto-Reply
2	Unscoped account lookup	4.2 Tenant-Scoped Lookup
3	No dedup on rapid re-ingest	4.1 (idempotency in queue)
4	Silent DB error suppression	4.4 Error Logging
5	No retry on API failure	4.1 Async Auto-Reply
6	Silent private reply failure	4.4 Error Logging
7	Sync ignores page settings	4.3 Unified Settings
8	Offset pagination degrades	4.6 Cursor Pagination
9	Unbounded reply preload	4.6 Bounded Replies
10	No status validation	4.7 Status FSM
11	WhatsApp bridge unscoped	4.5 Tenant Scope DB
12	Reply handler uses a.DB	4.5 Tenant Scope DB
13	No test for <code>pickRandomReplyText</code>	(Test coverage task)
14	<code>AppSecret</code> empty accepts webhook	(Already fixed per memory)
15	No merge conflict resolution	4.8 Frontend Conflicts

Table 5: Complete gap-to-enhancement traceability matrix.

6. Proposed Enhanced Sequence

The enhanced sequence moves auto-reply out of the synchronous webhook path and into an async worker. The webhook handler becomes faster and more resilient. The worker processes replies with built-in retry and proper error logging.

Actor	Action	Notes
Facebook API	POST /webhook with comment event	HMAC-SHA256 signed payload
Webhook Handler	Verify signature	Return 401 if invalid
Webhook Handler	Parse entry, extract page_id	
Webhook Handler	Tenant-scoped account lookup	Enhancement 4.2: org-scoped
Webhook Handler	Upsert comment (org-scoped DB)	Idempotent via ON CONFLICT
Webhook Handler	Enqueue auto-reply job to Redis Stream	Enhancement 4.1: async
Webhook Handler	Broadcast WebSocket (TypeCreated)	Org-scoped broadcast
Webhook Handler	Return HTTP 200 to Facebook	Fast response, no blocking
Queue Worker	Dequeue auto-reply job	Separate goroutine
Queue Worker	Resolve settings (org + page level)	Enhancement 4.3: unified
Queue Worker	Check auto-reply eligibility	Skip admin replies
Queue Worker	Send comment reply via Graph API	With retry + backoff
Queue Worker	Send private DM if enabled	Enhancement 4.4: log errors
Queue Worker	Store reply record (org-scoped DB)	Enhancement 4.5: scoped
Queue Worker	Send WhatsApp notification if enabled	Enhancement 4.5: scoped

Queue Worker	Broadcast WS (TypeReplyCreated)	Notify frontend of reply
--------------	---------------------------------	--------------------------

Table 6: Proposed enhanced processing sequence.

7. Implementation Priority Order

Implementations should follow the priority order below, grouped into three phases. Each phase should be fully tested and deployed before the next begins.

Phase	Enhancement	Priority	Rationale
1 - Security	4.2 Tenant-Scoped Lookup	Critical	Cross-tenant data leakage risk
1 - Security	4.5 Tenant Scope DB Access	Critical	Multi-tenant data isolation
2 - Reliability	4.4 Error Logging	High	Silent failures hide bugs
2 - Reliability	4.1 Async Auto-Reply	High	Webhook latency + retry
2 - Reliability	4.3 Unified Settings	Medium	Sync/webhook consistency
3 - Polish	4.7 Status FSM	Medium	Data integrity guard
3 - Polish	4.6 Cursor Pagination	Low	Performance at scale
3 - Polish	4.8 Frontend Conflicts	Low	User experience

Table 7: Three-phase implementation priority order.

8. Risk Assessment

Each enhancement carries implementation risks that should be mitigated through testing and gradual rollout:

8.1 Enhancement 4.1 (Async Auto-Reply)

Introducing a queue worker adds infrastructure complexity. Mitigation: reuse the existing Redis Streams infrastructure in `internal/queue/`. The worker should be idempotent: if a job is processed twice (e.g., due to a consumer crash), the second attempt must not create a duplicate reply. Use the comment ID + reply

text hash as a deduplication key.

8.2 Enhancement 4.2 (Tenant-Scoped Lookup)

Adding org-scoping to the page lookup may break webhook processing if a page is not yet associated with any organization. Mitigation: during the transition, fall back to the current unscoped query with a loud warning log, then tighten to scoped-only after confirming all pages are registered under their correct organizations.

8.3 Enhancement 4.5 (Tenant Scope DB)

Replacing `a.DB` with scoped DB requires verifying that all pre-condition queries (like finding the comment by ID) also respect tenant scoping. Mitigation: add integration tests that verify cross-org access is denied after the change.

8.4 Enhancement 4.6 (Cursor Pagination)

Changing the pagination API is a breaking frontend change. Mitigation: implement cursor pagination as a new parameter (`?cursor=xxx`) that coexists with the existing offset parameter during a deprecation period. Update the frontend service layer to prefer cursors while maintaining backward compatibility.

9. Files Affected by Enhancement

The following table lists every file that will be modified by the proposed enhancements, along with the expected change type.

File	Enhancement(s)	Change Type
internal/handlers/fb_comments.go	4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7	Major refactor
internal/models/fb_comment.go	4.2, 4.7	Add unique index, FSM model
internal/worker/	4.1	New worker consumer
internal/queue/	4.1	New stream definition
frontend/src/services/facebookComments.ts	4.6, 4.8	Cursor pagination API
frontend/src/views/facebook/facebookCommentsMerge.ts	4.8	Conflict resolution logic
internal/handlers/fb_comments_test.go	All	Expanded test coverage

Table 8: Files affected by each enhancement.

10. Conclusion

The Facebook comments feature in Whatomate is functionally complete but harbors several critical issues around multi-tenant isolation and operational observability. The two critical-priority fixes (tenant-scoped account lookup and scoped DB access) should be addressed immediately as they represent potential cross-tenant data leakage vectors.

The async auto-reply enhancement (4.1) is the largest single change but provides the most operational benefit: faster webhook response times, built-in retry logic, and natural separation of concerns. It should follow the security fixes in Phase 2.

The remaining enhancements (cursor pagination, status FSM, frontend conflict resolution) are quality-of-life improvements that can be scheduled for a subsequent release cycle. All changes maintain backward compatibility and follow the existing plugin/extension architecture patterns established in the Whatomate codebase.